RIPHAH INTERNATIONAL UNIVERSITY

Federally Chartered University, Recognized by HEC

GAME PROGRAMMING

Assignment: 03

Date of submission: 7 June, 2026

Submitted by: Muhammad Anas
Semester: BsCs-5
SAP: 59904

1. Game Overview

Among Us is a 2D online multiplayer game for 4 to 15 players. At the start of a match, players are secretly split into two teams: Crewmates and Impostors.

Crewmates win by completing all their tasks or voting out all the killers. Impostors win by sneaking around vents, sabotaging the ship and eliminating crewmates until they have equal numbers. The game shifts back and forth between exploration and voting discussions.

2. High-Level Architecture

To keep the game fair and fun, the server must be in total control. This is called a server-authoritative design. If you let individual players decide if they are alive or dead, people can easily cheat.

The main systems you need to build are:

- Network Manager: Creates game lobbies using room codes, connects players together and syncs settings like player speed.
- Match State Machine: Controls the flow of the game. It switches everyone at the same time from the lobby to the match, then to meetings and finally to the game over screen.
- Player Tracker: Manages who is alive, who is dead, and who is an impostor. It hides impostor details from the crewmates.
- Task System: Tracks each crewmate's progress bar and triggers the global win condition when the bar reaches 100 percent.
- Interaction Handler: Checks if a player is close enough to use a vent, report a body, or eliminate another player.

3. Code Structure

A great way to organize the code is by using a component-based structure. This means characters are game objects, and we plug different scripts into them to give them behaviors.

Main Scripts and Responsibilities:

- GameManager: This runs on the server. It acts as the brain of the match and knows the true state of every player and task.
- PlayerController: This sits on the client side. It listens to keyboard or screen joystick input so the player can move around.
- CharacterStats: A script attached to each player that holds basic data like current room, visibility range, and life status.
- SabotageSystem: Manages global emergencies like turning off the lights or starting a countdown for a reactor meltdown.

How Systems Talk to Each Other:

Instead of scripts constantly calling each other directly, you should use events. For example, when an impostor clicks the kill button, the PlayerController sends a request to the server. The server verifies the distance and cooldown. If everything checks out, the server changes the victim's status to dead and broadcasts a message to all players to update their screens.

4. Design Decisions

You need to justify these choices in your document. Here are three clear reasons for this setup:

- Anti Cheat Security: By keeping all major decisions on the server, a hacked client cannot instantly kill everyone or view who the impostor is. The server only shares information that a player is supposed to see.
- Clean State Changes: When someone reports a body, the game needs to instantly stop movement, teleport everyone to the table and open the chat UI. Using a clear state machine makes sure no player gets left behind or glitched in a wall during the transition.
- Coding Flexibility: Using components makes it easy to handle ghosts. When a crewmate dies, you do not delete their character. You simply turn off their physical collision component and change their visual component to look transparent.